IEEE Access
Multidisciplinary ⋮ Rapid Review ⋮ Open Access Journal

Date of publication xxxx 00, 0000, date of current version March 11, 2026.

Digital Object Identifier 10.1109/ACCESS.2017.DOI

Resource-Constrained Evaluation of Quantized Local LLMs for Retrieval-Augmented Generation

A[1], B[1]
[1]c

Corresponding author: b.

ABSTRACT This paper studies retrieval-augmented generation (RAG) under a realistic local deployment constraint using a single consumer workstation with one 24 GB VRAM GPU. The goal is not to propose a new retriever or generator architecture, but to evaluate how local, quantized RAG behaves when answer quality, provenance recovery, semantic adequacy, latency, and runtime robustness are treated as joint outcomes. Using a reduced KILT-NQ setup, the study compares 12 local generators across multiple scales and model families, 3 embedding models, and 4 pipeline conditions: retrieval-only evaluation, closed-book generation, standard retrieve-then-answer RAG, and same-model query rewriting followed by retrieval-augmented answering.

The results show that retrieval-augmented answering substantially improves over closed-book generation across the full generator set, but the best local configuration cannot be inferred from model size alone. Retriever choice remains important not only for provenance recovery but also for end-to-end deployability. Same-model query rewriting is not justified in the present setup, because it increases latency and failure exposure without surpassing standard RAG in lexical, grounded, or semantic performance. Semantic answer similarity (SAS) broadly correlates with lexical quality, but also reveals informative outliers.

Together, these findings show that local RAG is not simply a smaller version of cloud-scale benchmarking, but a distinct evaluation regime in which retrieval quality, grounding, semantic adequacy, latency, and robustness must all be measured explicitly. The paper argues that single-workstation studies are scientifically useful in their own right and should be treated as a necessary complement to larger-scale, cloud-oriented evaluation.

INDEX TERMS Retrieval-Augmented Generation, RAG, quantized inference, local large language models, LLM, open-domain question answering, provenance-aware evaluation, dense retrieval, query rewriting, resource-constrained deployment, grounded QA.

I. INTRODUCTION

LARGE Language Models (LLMs) excel at producing fluent, smooth, natural-sounding text. However, they can struggle with rare or unusual factual questions, topics outside their training area, or information that has changed or been updated recently. Retrieval-Augmented Generation (RAG) is a commonly used method to address this problem. In this paradigm, the generator no longer relies only on parametric memory. Instead, it uses information retrieved from external sources to support its answers. This allows the model to use up-to-date information without needing retraining and, in principle, makes it easier to inspect the basis of the answer [1]. Since the original RAG formulation, the research community has documented both clear improvements and ongoing problems in RAG pipelines. For example, irrelevant retrieved information can confuse the model, missing evidence can trigger hallucinated answers, and the modular nature of RAG makes it difficult to pinpoint whether failure comes from the retriever, the embedding model, the chunking/indexing design, or the generator itself [2]–[5].

On the other hand, real-world deployment trends are shifting evaluation priorities. On-device or near-device execution is growing, for example, due to privacy requirements, lower deployment cost, or latency constraints. Because of this, people increasingly use quantization and smaller, more efficient inference setups to run LLMs locally [3], [6]. However,

much of the current RAG evaluation literature, including large benchmark suites and modular libraries, still implicitly assumes server-side inference, managed services, or cloud and cluster resources. It also often treats inference as "best effort," where occasional runtime issues are not counted as part of the results [7]. This creates a mismatch for researchers who want to reproduce results under realistic compute ceilings, such as a single consumer GPU, and for practitioners who must choose among many local quantized models under strict memory and latency budgets.

This mismatch is especially important because controlled end-to-end evaluations of local RAG systems are still limited. Recent surveys and benchmarking studies describe RAG evaluation as fragmented and highlight the need for more systematic and reproducible comparisons across components, metrics, and deployment settings [7], [8] . At the same time, interest in efficient, private, and edge-oriented RAG is growing, including edge-assisted, distributed, and federated approaches [9]–[11]. However, much of this work examines architecture, privacy, latency, serving strategy, or application-specific performance separately, rather than comparing multiple local, quantized generators end-to-end under the same fixed conditions and treating system trade-offs as part of the result [12], [13]. For small labs, individual researchers, and engineers building private or low-cost systems, this gap is significant because it shapes which models and RAG designs are actually reproducible and practical to deploy.

Our main motivation is therefore to study RAG systems locally. Rather than proposing a new retriever, a new benchmark, or a new LLM architecture, we study how local, quantized generators behave in a reproducible RAG workflow under explicit constraints. Accordingly, this paper positions its contribution as a resource-constrained evaluation framework for grounded QA using local RAG. Our framework uses a single GPU, quantized local models served with Ollama, embedding-based retrieval locally with FAISS, evaluates answers and their sources using KILT-style metrics, and also tracks and reports runtime failures explicitly. This study is generator-centric, where we compare multiple local generators across different scales and families while keeping the task and hardware conditions fixed.

Our second main motivation is accessibility. In today's LLM research, many results are still hard to reproduce unless one has access to substantial cloud computing or large-scale institutional infrastructure. In contrast, this work shows that research can still produce meaningful, reproducible results on a single consumer workstation. The goal is not to argue against larger systems, but to show that careful, reliable experiments are still possible with limited hardware, while remaining scientifically useful for exposing trade-offs that may be hidden in cloud-first evaluation. This matters especially for researchers and practitioners who need to decide not only which configuration is most accurate, but which one is stable, efficient, and realistic to run locally.

This study is also motivated by a methodological observation that a model may fail not only by generating an incorrect answer, but also by stalling, or failing to emit a first token. Standard benchmarks often ignore these runtime failures, but for local use they can be decisive. Recent engineering reports on RAG systems also note that reliability issues often appear during real operation and change as the system design changes, instead of being fully solved from the start [14].

Given this context, this paper makes four main contributions: (i) it presents a reproducible, single-GPU framework to evaluate RAG systems that use quantized local LLMs under explicit memory and runtime constraints; (ii) it proposes a generator-centric evaluation design that separates retrieval quality, closed-book generation (no retrieval), standard RAG, and same-model query rewriting; (iii) it expands evaluation beyond answer quality by incorporating retrieval performance, provenance-aware lexical metrics, semantic answer similarity, latency, and runtime failure; and (iv) it argues that budget-accessible workstation experiments can still be scientifically valuable, especially for edge-oriented RAG research.

The remainder of the paper is organized as follows. Section II reviews the technical background on retrieval-augmented generation, provenance-aware evaluation, dense retrieval, quantization, and local deployment constraints. Section III situates the work relative to previous research on RAG, query rewriting, retrieval benchmarking, efficient LLM inference, and emerging local or edge-oriented RAG systems. Section IV presents the methodology, including the research questions, dataset choices, model selection, execution pipelines, and evaluation protocol. Section V presents the experimental results, covering retrieval performance, end-to-end question answering, lexical and semantic evaluation, query rewriting, latency, and runtime robustness under local workstation constraints. Section VI discusses the implications of these findings, their practical relevance for local RAG deployment, and the main limitations of the current study. Finally, Section VII concludes the paper by summarizing its main contributions and reflecting briefly on the implications of the results for local, resource-constrained RAG evaluation.

II. BACKGROUND

This section addresses the key theoretical concepts of our study: the limits of parametric memory in LLMs and the motivation for retrieval-augmented generation, the standard modular RAG pipeline (dense retrieval, indexing, and query rewriting), quantization as the enabler for single-GPU local inference, and provenance-aware evaluation for separating answer correctness from evidence grounding under local reliability constraints.

A. FROM PARAMETRIC MEMORY TO RETRIEVAL-AUGMENTED GENERATION

Large language models store a surprising amount of "world knowledge" inside their parameters because of how they are trained. During large-scale self-supervised pretraining, the model repeatedly learns to predict missing text using vast amounts of mostly unlabeled text, either by predicting

the next token in a sequence (autoregressive training) or by predicting masked tokens in context [15].

In both cases, training minimizes a token-level negative log-likelihood. For autoregressive pretraining $\mathcal{L}_{\mathrm{AR}}(\theta)$ on a length-T token sequence, the objective is:

$$\mathcal{L}_{\mathrm{AR}}(\theta) \;=\; -\sum_{t=1}^{T} \log p_\theta(x_t \mid x_{<t})\,. \tag{1}$$

For masked-token pretraining $\mathcal{L}_{\mathrm{MLM}}(\theta)$, the loss is computed only on masked positions:

$$\mathcal{L}_{\mathrm{MLM}}(\theta) \;=\; -\sum_{t\in M} \log p_\theta\big(x_t \mid x_{\setminus M}\big)\,, \tag{2}$$

where θ are the trainable parameters, M is the set of masked indices, x is the token sequence, and $x_{\setminus M}$ denotes the sequence with masked tokens replaced by placeholders.

In modern pipelines, that text often comes from internet-scale corpora assembled from web crawls plus other large text sources, so the model is effectively exposed to a broad slice of online language and facts during training [16]. Over many gradient updates, frequently repeated patterns (such as names, relations, and events) become encoded in a distributed way across weights, which is why these pretrained language models can already answer factual prompts without seeing any supporting document at inference time [17].

However, what ends up being stored is not uniform. Information that occurs infrequently in the pretraining corpus (i.e., long-tail facts) is much less likely to be learned consistently. Therefore, model accuracy tends to scale with the amount of relevant supporting evidence available during pretraining [18].

This parametric memory is also hard to update. Changing a specific fact typically requires additional training or specialized model-editing methods, which can be costly and may cause unwanted side effects on other behaviors [19]. Finally, because this knowledge is not stored explicitly but as entangled parameters, it is difficult to inspect or justify where a particular answer came from. As a result, when answering knowledge-intensive questions, the model may produce confident hallucinations and other unsupported outputs [17].

Retrieval-Augmented Generation (RAG) emerged to compensate for the limits of parametric memory in LLMs. Instead of relying exclusively on what is stored in model weights, a RAG system retrieves relevant text from an external corpus at inference time and then conditions the generator on that retrieved evidence. In the original RAG formulation by Lewis et al. [1], retrieval is not an optional post-processing step but a core mechanism of the model's access to knowledge.

Formally, generation can be written as a mixture over retrieved passages:

$$p(y \mid x) \;\approx\; \sum_{i=1}^{k} p_\eta(z_i \mid x)\, p_\theta(y \mid x, z_i), \tag{3}$$

where x is the input query, z_i are retrieved passages, $p_\eta(z_i \mid x)$ is the retriever distribution, and $p_\theta(y \mid x, z_i)$ is the generator conditioned on a passage. A common parameterization of the retriever distribution uses a softmax over similarity scores:

$$p_\eta(z_i \mid x) \;=\; \frac{\exp(s_\eta(x, z_i))}{\sum_j \exp(s_\eta(x, z_j))}. \tag{4}$$

The practical value is that, as long as the system can fetch the right supporting passages, even a smaller or moderately sized generator can produce grounded answers without requiring the entire knowledge source to be compressed into its parameters, and without retraining whenever the underlying corpus changes.

Since that initial formulation, RAG systems have increasingly shifted from their original end-to-end formulation toward a more modular, engineering-friendly pipeline. In most practical systems, the knowledge corpus is first chunked into passages, each passage is embedded into a dense vector space (often using a dual-encoder retriever), and those vectors are stored in an approximate nearest-neighbor index for fast similarity search. At inference time, the user query is embedded in the same space, the system retrieves the top-k most similar passages, and these passages are then inserted into the prompt of the generator LLM (sometimes with an additional reranking or formatting step) [1]. This RAG pipeline has become a de facto standard because it is relatively easy to implement, can be updated without retraining, and is compatible with both open-weight and closed-weight models. Retrieval can be local and open, while the generator may be API-served.

The success of modern RAG systems depends heavily on dense retrieval, especially Dense Passage Retrieval (DPR), a dual-encoder approach that embeds the query and each passage into the same vector space, then retrieves by vector similarity. It is helpful to contrast this with lexical retrieval (sparse), where queries and documents are represented as high-dimensional sparse term vectors and scored largely by exact term overlap. Dense retrievers, on the other hand, project passages and queries into a shared embedding space and retrieve passages by nearest-neighbor search. This strategy has become central to open-domain QA and practical RAG because it is robust to paraphrase, supports efficient vector indexing, and scales well to large corpora [20].

DPR can be trained and improved largely independently of the generator, while vector-search toolchains (such as FAISS) make large-scale passage lookup efficient even on commodity hardware [21].

In the standard dual-encoder formulation, a query encoder f_q and passage encoder f_p map text to d-dimensional vectors:

$$\mathbf{q} = f_q(x), \qquad \mathbf{p}_i = f_p(z_i), \qquad \mathbf{q}, \mathbf{p}_i \in \mathbb{R}^d. \tag{5}$$

Similarity is typically computed via inner product or cosine similarity:

$$s(x, z_i) = \frac{\mathbf{q}^\top \mathbf{p}_i}{\|\mathbf{q}\|_2\, \|\mathbf{p}_i\|_2} \quad \text{(cosine)}, \tag{6}$$

and retrieval returns the top-k passages by approximate nearest-neighbor search:

$$R_k(x) = \mathrm{TopK}_i\, s(x, z_i). \tag{7}$$

In practice, this made RAG systems easier to engineer: embeddings can be precomputed, stored in a fast nearest-neighbor index, and swapped or refreshed without retraining the generator, while also improving recall when relevant passages do not share exact keywords with the query. This modular pipeline is also easy to update. One can add new documents, re-embed and re-index the corpus, or swap retrievers and generators without retraining an end-to-end RAG model [22].

Another important ingredient in practical RAG is query reformulation. User questions are often phrased in ways that are semantically clear to humans but suboptimal for retrieval. Recent work has shown that reformulating the query before retrieval can improve evidence recall and downstream generation quality [3], [23]. Conceptually, rewriting acts as a bridge between natural user phrasing and the wording that best exposes relevant passages in the indexed corpus.

In pipeline terms, rewriting composes a rewrite function with the retriever:

$$\tilde{x} \;=\; r_\phi(x), \qquad R_k(x) \;=\; \mathrm{TopK}_i\, s(\tilde{x}, z_i), \tag{8}$$

where r_ϕ maps the user query x to a retrieval-oriented query $\tilde{x}$, and retrieval is performed using $\tilde{x}$ rather than the original query.

This issue is particularly relevant in local RAG because retrieval errors can be amplified by smaller quantized generators. If the retrieved passages are weak or off-topic, a weaker generator may be unable to recover and may either answer poorly or hallucinate.

Accordingly, this paper studies the standard modular RAG pipeline under strict local workstation constraints, where performance depends not only on the generator but also on passage chunking, embedding quality, indexing and search settings, and context budgeting. We therefore evaluate retrieval both for its contribution to answer quality and for its deployment cost. Finally, because query phrasing can strongly affect retrieval, especially when a quantized generator cannot easily recover from weak evidence, we include query rewriting as a condition produced by the same local generator in order to avoid adding extra system complexity.

B. QUANTIZATION AS AN ENABLER OF LOCAL LLM RESEARCH

The feasibility of this study depends on advances in quantized inference, which compresses a model's parameters from floating-point formats (e.g., FP16) into lower-bit representations so they fit, and run efficiently, on a single consumer GPU. In practice, quantization replaces each weight tensor with low-bit integers and a small set of per-block (or per-channel) scale and offset values that approximately reconstruct the original weights during matrix multiplications.

A common view is blockwise affine reconstruction of the original weight values: for weights in a block, we store a low-bit integer code plus block statistics (scale and offset), then reconstruct approximately at multiply time. One convenient way to write this is:

$$q \;=\; \mathrm{clip}\left(\left\lfloor \frac{w-m}{s} \right\rceil,\; q_{\min},\; q_{\max}\right), \qquad \hat{w} \;=\; s\cdot q + m, \tag{9}$$

where $\lfloor\cdot\rceil$ denotes rounding to the nearest integer, clip enforces the representable integer range, $\hat{w}$ is the dequantized approximation used in computation, q is the quantized representation, and s, m are per-block scale and offset used during dequantization.

Because local feasibility is largely a memory question, it is also useful to relate the effective bits-per-weight to model footprint:

$$\mathrm{ModelBytes} \;\approx\; \frac{N_{\mathrm{w}}\, b_{\mathrm{eff}}}{8}, \tag{10}$$

where N_{w} is the number of stored weights and b_{eff} is the effective bits-per-weight including format overhead.

Recent work on efficient post-training quantization, including GPTQ/OPTQ, SmoothQuant, AWQ, and related methods, has made it realistic to deploy multi-billion-parameter models on consumer hardware while retaining much of their utility [6], [24], [25].

Our experiments primarily use Q4_K_M, a widely deployed 4-bit K-quant "mix" introduced with the llama.cpp k-quants design.

Our experiments primarily use Q4_K_M, a widely deployed 4-bit K-quant "mix" introduced with the llama.cpp k-quants design. It is the dominant generator format in this study, but not the only one. Some models use different deployment formats exposed by their local distribution (for example, GPT-OSS:20B uses MXFP4), and the embedding models span both F16 and Q4_K_M.

At the weight-format level, Q4_K quantizes weights in super-blocks of 256 values, subdivided into smaller sub-blocks (commonly 8×32), with block-local scales and offsets also stored in quantized form; this hierarchical structure yields an effective footprint of about 4.5 bits per weight once metadata overhead is accounted for. The suffix _M ("medium") denotes a mixed-precision format rather than a single uniform tensor type. In the reference implementation, it uses mostly 4-bit K-quant weights, but selectively upgrades particularly sensitive tensors (notably parts of attention) to higher precision (e.g., Q6_K) to reduce error where it matters most, improving quality at only a modest size increase [26], [27].

In this paper, quantization is therefore not just a convenience. It is part of the scientific object of study. A quantized model may fit in memory yet still exhibit poor latency, weak evidence use, or unstable behavior under multi-stage prompting. Conversely, a smaller model may be highly deployable but too weak to benefit from retrieval. Under a fixed local hardware budget, the relevant question is not only whether a model fits, but how quantization changes the practical trade-off among answer quality, latency, and robustness.

C. PROVENANCE-AWARE EVALUATION IN LOCAL RAG

In knowledge-intensive Question Answering (QA), it is often insufficient to ask only whether the model produced the right answer string. A model may answer correctly because it retrieved the right evidence, because it relied on parametric memory, or because it happened to guess correctly. From a scientific standpoint, these cases should not be treated as equivalent. Provenance-aware evaluation addresses this by checking not only what was answered, but also whether the answer is supported by the retrieved sources. The KILT benchmark was introduced in part to make this distinction explicit by grounding multiple knowledge-intensive tasks in a shared Wikipedia snapshot and incorporating provenance into evaluation [28].

For a study of local RAG, this provenance-aware perspective is especially important: if answer quality improves, it matters whether that improvement comes from better retrieval, better use of retrieved evidence, or simply from the internal knowledge of the generator. For these reasons, KILT-NQ is a natural fit for the present setting. It preserves the realism of open-domain QA while allowing us to evaluate whether retrieved passages were relevant and whether correct answers were grounded in those passages [28].

At the same time, answer quality in local RAG should not be assessed only through lexical overlap. Exact Match (EM) and token-level F1 remain useful because they are standard, interpretable, and compatible with provenance-based evaluation. But semantically correct answers may still be phrased differently from the reference, especially when comparing generators of different sizes, architectures, or prompting behavior. For that reason, recent RAG evaluation work increasingly distinguishes between lexical agreement, semantic correctness, and evidence-grounded correctness rather than treating them as the same thing [29]. In the present setting, this distinction matters because a local model can fail in more than one way. It may produce an unsupported answer, generate a lexically different but semantically valid response, or fail to complete inference reliably.

Running RAG locally also changes what counts as a meaningful result. In server-side evaluations, it is often assumed that a selected model will eventually return an output for every test case, and that runtime instability is not part of the outcome. Under local deployment, this assumption can break down. Model loading overhead, VRAM pressure, prompt length, serving-layer interactions, and quantized-runtime quirks can all lead to stalls, timeouts, or partially completed runs. These failures are easy to dismiss as implementation noise, but from a local deployment perspective they are often decisive [30], [31].

For this reason, the present work treats local infrastructure as part of the methodology rather than as a hidden detail. Generation is served locally through Ollama [32], retrieval is implemented with FAISS [21], experiment outputs are written incrementally, and timeout policies are explicitly recorded. This design reflects the reality that in single-GPU research, robustness is inseparable from utility. A model that is marginally more accurate but frequently fails to emit a first token within a practical budget may be less valuable than a slightly weaker but stable alternative.

Taken together, the background for this work is not only RAG as an NLP technique. It is RAG as a local, constrained, deployment-aware scientific problem. A problem in which provenance, semantic adequacy and operational reliability must be evaluated side by side.

III. RELATED WORKS

This study is best situated not in the broadest literature on retrieval-augmented generation, but in the more specific line of work that made modular open-domain QA with explicit evidence possible.

In the broader shift from purely parametric question answering (QA) to retrieval-augmented generation, REALM showed that retrieval could be integrated into language-model training as a mechanism for external knowledge access [5], DPR established dense dual-encoder retrieval as a strong practical method for open-domain QA [20], and RAG formalized the retrieve-then-generate paradigm in which a generator is conditioned on passages retrieved at inference time rather than relying only on parametric memory [1]. For our paper, KILT is especially important because it reframed evaluation around a shared Wikipedia snapshot and explicit provenance, making it possible to distinguish answer correctness from evidence-grounded correctness [28]. WResearch on generative readers that combine evidence from several retrieved passages has also shown that retrieval depth can significantly influence downstream question-answering performance. This supports treating top-k as an experimental variable rather than a small implementation detail [33].

The papers most relevant to our work are those that consider retrieval quality to be a separate component of a modular RAG pipeline. Query rewriting has been shown to improve retrieval by reformulating user questions into retrieval-oriented queries that better match the indexed corpus [23]. Related retrieval-focused methods, such as HyDE and generation-augmented retrieval, similarly show that model-generated intermediate text can improve evidence recall before the final answer is produced. [34], [35]. At the same time, benchmark studies such as BEIR and MTEB warn against assuming that one embedding model is always the best. Retrieval performance varies widely across datasets, domains, and task types, so the choice of retriever should be treated as an experimental variable rather than fixed background infrastructure [36], [37]. This concern is closely related to recent evaluation research showing that modular RAG systems should be assessed by individual components, not only by end-task accuracy. Reports from real-world deployments describe recurring retrieval and system failure modes [38], while frameworks such as ARES and RAGChecker explicitly evaluate retrieval and generation separately [39], [40]. More recent grounding benchmarks such as GaRAGe and GroUSE further show that even strong systems and evaluators can miss important grounding failures.

This reinforces the need for provenance-aware evaluation beyond raw answer quality [14], [41].

The systems context most closely related to this paper is the growing body of research on efficient local inference and retrieval augmentation under limited resources. Quantization methods such as LLM.int8(), GPTQ/OPTQ, and AWQ have greatly reduced the memory requirements for running large generators locally, making inference with multi-billion-parameter models possible on more modest hardware [24], [42]. Systems research such as PowerInfer pushed this further by focusing directly on local LLM serving on consumer-grade GPUs [43]. At the same time, newer local and on-device retrieval systems, such as MeMemo, EdgeRAG, and RAGDoll, show growing interest in privacy-preserving, edge-based, and single-GPU forms of retrieval augmentation [44]–[46]. Recent domain-specific studies also show that local RAG can be useful in privacy-sensitive settings, for example by improving models that are deployed locally in clinical workflows [47]. However, these lines of research usually optimize serving, indexing, or application performance separately. This paper instead studies where they overlap. It provides a reproducible, provenance-aware evaluation of local, quantized generators within a modular RAG workflow under clear workstation constraints, while treating latency, robustness, and runtime failures as part of the scientific results rather than as incidental engineering noise.

IV. METHODOLOGY

This work evaluates local, single-GPU Retrieval-Augmented Generation (RAG) as a deployable system under a fixed hardware environment. The methodology is generator-centric. The primary sweep varies the local, quantized generator because it most strongly determines VRAM use, latency, and runtime stability in realistic local deployment. The retriever is still varied (multiple embedding models and two retrieval depths) to expose a practical cost–benefit landscape rather than a single "best" configuration.

A. RESEARCH QUESTIONS

This study was structured around four research questions:

1) **RQ1 — Retriever impact.** How do embedding model choice and retrieval depth (top-$k = 5, 10$) affect provenance retrieval quality and retrieval cost in the local setup?
2) **RQ2 — Generator impact.** With retrieval held fixed, how do local quantized generators differ in answer quality and latency?
3) **RQ3 — Single-model query rewriting.** Does using the same local generator to rewrite the query improve end-to-end QA enough to justify the added step? And how does the effect vary across local generators?
4) **RQ4 — Robustness under local constraints.** Which runtime failures dominate under a single-GPU local deployment budget, and how strongly do they affect the practical usefulness of each pipeline?

Together, these questions evaluate local RAG as a deployable system rather than an abstract benchmark. We isolate retrieval quality with RQ1, we quantify generator differences under fixed retrieval with RQ2, we test whether self-rewriting is worth the extra step with RQ3, and we characterize robustness and dominant failure modes under a single-GPU budget with RQ4.

B. TASK, DATA, AND SETUP

The primary task is open-domain question answering on KILT-NQ [28], chosen for its Wikipedia-based knowledge source and provenance-aware evaluation. For the scope of this paper, we use a deterministic subset of 500 development questions to enable repeated local experiments and early Pareto analysis on a single workstation. The subset is sampled in random mode with a fixed seed of 123.

The retriever is evaluated with three local embedding models chosen to represent distinct practical operating points [48]. Retrieval depth is evaluated at $k = 5$ and $k = 10$.

To keep repeated runs feasible locally, the knowledge base is a reduced corpus. It includes all gold provenance passages for the 500 questions, plus 100,000 randomly selected passages from the same KILT Wikipedia snapshot. This keeps the task open-domain while making indexing and repeated retrieval easier to handle. Passages and metadata are stored in SQLite, and dense indices are built per embedding model using FAISS [21]. The corpus is built once and reused across experiments to avoid recomputation and improve reproducibility.

The overall experiment is intentionally designed for a fixed single-GPU execution rather than for cloud or cluster. In that sense, the reduced corpus is not merely a convenience. It is part of the controlled local setting that makes repeated generator comparisons, latency measurement, and failure analysis possible under realistic hardware limits. In total, the reported experiment workload required over 160 GPU-hours of execution on a local machine. This figure reflects experiment GPU-time only; it excludes additional project overhead such as pre-flight checks, debugging, reruns, version updates, and model downloads.

C. MODELS, PIPELINES, AND CONTROLLED SETTINGS

The models are served locally through Ollama [32]. The primary generator sweep spans six quantized Qwen3 [49] local models at approximately 0.6B, 1.7B, 4B, 8B, 14B, and 30B parameters, covering from edge-like up to the upper bound that still fits (quantized) on a 24 GB VRAM GPU. We adopt Qwen3 specifically because it provides a single-family, multi-scale ladder, including both dense and mixture-of-experts variants, that supports controlled within-family comparisons across model scale without changing the overall family and training lineage [49].

To test whether the observed patterns are overly specific to that family, we also include a selection of comparison models from other families: LFM2.5:1.2B, Gemma3:4B, Granite4:7B, LFM2:8B, Gemma3:12B, and GPT-OSS:20B

[50]–[54]. These additional models are used as targeted comparison points rather than as a full inter-family benchmark. The resulting generator set therefore combines a controlled Qwen3 within-family scaling ladder with a broader selection of local deployment models from different model families.

Table 1 summarizes the 12 generator checkpoints used in this study, grouped into deployment-oriented tiers. For each model, we report the Ollama tag, approximate parameter scale, architecture type, the quantization variant, and the corresponding on-disk model size, together with a brief selection rationale.

We evaluate three local embedding models that reflect different real-world performance and resource trade-offs. A compact popular baseline, a modern mid-scale local embedder, and a stronger larger embedder. The goal is not to cover every option, but to include enough variation to test whether the choice of retriever changes the downstream results.

Table 2 captures three deliberately different operating points for local retrieval. Qwen3-Embedding-4B is tagged as a "strong" embedder because it ranked among the top performances on embedding and retrieval benchmarks [48], [55]. All-MiniLM-L6-v2 is included as a compact, widely used Sentence-BERT–style baseline [37], [56]. Nomic-embed-text-v1.5 provides a modern mid-scale point. Nomic Embed is a 137M-parameter long-context text embedding model designed for retrieval tasks and intended for RAG-style use cases [57].

We tested and evaluated the local embedding and generator models under four pipelines:

- **P0 (Retrieval-only):** retrieve top-k passages, no answer generation (isolates retriever behavior).
- **P1 (Closed-book):** generator answers without retrieval (baseline without external evidence).
- **P2 (Standard RAG):** retrieve using the original question, then answer using the top-k retrieved passages.
- **P3 (Self-rewrite RAG):** the same generator rewrites the question into a retrieval-oriented query, retrieves, then answers using the retrieved passages.

This structure supports direct diagnosis aligned with the research questions. P0 isolates the retriever and helps answer RQ1 by measuring provenance retrieval quality. P1 vs. P2 tests whether retrieved evidence actually improves QA. In other words, it tests whether the system benefits from retrieval in an end-to-end sense, directly supporting RQ2 and helping interpret RQ1. P2 vs. P3 isolates the incremental value of single-model query rewriting, directly answering RQ3. Finally, divergences such as strong P0 but weak P2, or elevated runtime failures in any pipeline, indicate generation-side or serving-side bottlenecks and are analyzed under RQ4.

The controlled variables are: generator model, embedding model, retrieval depth ($k = 5, 10$), and presence or absence of query rewriting. Context length is fixed at 4096, and decoding settings are held fixed across generators to avoid per-model tuning confounds. In the reported generator summaries, the compact P2 and P3 tables show the best scores across the evaluated embedders for each generator and top-k setting.

Prompting is an important component of LLM-based pipelines because it defines the task framing presented to the model, including its role, output constraints, and permissible evidence source. Prior work has also shown that model performance can be sensitive to prompt formulation. Accordingly, the present study uses explicit system prompts matched to the function of each pipeline [58], [59].

In P1, we use a concise closed-book answering prompt. In P2, answer generation uses a context-restricted prompt. In P3, the generator first uses a retrieval-oriented query rewriting prompt and then uses the same context-restricted answering prompt as in P2:

- **Closed-book prompt**
  You are a helpful assistant. Answer the question concisely.
- **Context-restricted prompt**
  You are a helpful assistant. Answer the question using ONLY the provided context. If the answer cannot be found in the context, say: 'I don't know based on the provided context.' Be concise.
- **Query rewriting prompt**
  You are a search query rewriter for open-domain question answering. Rewrite the user question into a concise search query that is likely to retrieve relevant Wikipedia passages. Return ONLY the rewritten query, without quotes or extra text.

D. WORKSTATION

All experiments were conducted on a single personal workstation running Windows 10 (version 10.0), with an AMD64 Family 25 Model 33 CPU (32 physical cores, 32 threads), 64.0 GB of RAM, and a single NVIDIA GeForce RTX 3090 GPU with 24.0 GB of VRAM. This hardware configuration was used to represent a realistic high-end local setup rather than a multi-node or cloud environment. Local inference was served through Ollama version 0.17.1. Retrieval indexing and similarity search used FAISS version 1.13.2, and the orchestration code was implemented in Python 3.11.14. To improve reproducibility, all runs used a fixed random seed of 123.

E. EVALUATION METRICS

The evaluation covers answer metrics, retrieval metrics, provenance-constrained grounded metrics, semantic answer similarity, efficiency metrics, and robustness metrics. We report standard open-domain QA scores (exact match and token-level F1) together with provenance-aware retrieval measures (Recall@k and R-precision), provenance-constrained grounded scores, and semantic answer similarity (SAS). Quality metrics are computed for completed answer-producing trials. Failed trials are retained explicitly and analyzed under robustness rather than silently discarded.

Tier	Model (Ollama tag)	Params	Type	Quantization	Size	Rationale
Tiny	`qwen3:0.6b`	752M	Dense	Q4_K_M	523MB	Lower-bound baseline for tight constraints.
Tiny	`lfm2.5-thinking:1.2b`	1.17B	Hybrid	Q4_K_M	731MB	Reasoning-oriented small comparison model.
Small	`qwen3:1.7b`	2.03B	Dense	Q4_K_M	1.4GB	Upper limit for edge-like deployment.
Medium	`qwen3:4b`	4.02B	Dense	Q4_K_M	2.5GB	Mid-scale local Qwen3 baseline.
Medium	`gemma3:4b`	4.3B	Dense	Q4_K_M	3.3GB	Non-thinking lightweight comparison model.
Medium	`granite4:7b-a1b-h`	6.94B	Hybrid	Q4_K_M	4.2GB	Hybrid thinking model with 1B active parameters.
Medium	`qwen3:8b`	8.19B	Dense	Q4_K_M	5.2GB	Midrange Qwen3 quality/latency point.
Medium	`Maternion/lfm2:8b`	8.34B	MoE	Q4_K_M	5.0GB	MoE-hybrid-style mid comparison model.
Workstation	`gemma3:12b`	12.0B	Dense	Q4_K_M	8.1GB	Larger dense non-thinking comparison model.
Workstation	`qwen3:14b`	14.8B	Dense	Q4_K_M	9.3GB	High-capacity dense model that fits a single GPU.
Workstation	`gpt-oss:20b`	20.9B	MoE	MXFP4	14.0GB	Reasoning-oriented MoE comparison model.
Workstation	`qwen3:30b`	30.5B	MoE	Q4_K_M	19.0GB	Upper limit on a 24GB GPU. Best-case baseline.

TABLE 1. Generators used in this paper. Parameters, quantization, type, and size correspond to the local model tags used in the experiment configuration.

Tier	Model (Ollama tag)	Params	Quantization	Size	Rationale
Baseline	`all-MiniLM-L6-v2`	22.6M	F16	46MB	Compact baseline embedder.
Mid	`nomic-embed-text:v1.5`	137M	F16	274MB	Modern mid-scale long-context embedder.
Strong	`qwen3-embedding:4b`	4.02B	Q4_K_M	2.5GB	High-quality local embedder.

TABLE 2. Embedding models used in the retrieval stack. Parameters, quantization, and size correspond to the local model tags used in the experiment configuration.

1) Answer metrics: EM and token-level F1.

Let q denote an input question, let $\hat{a}$ denote the answer string produced by the model for q, and let $\mathcal{A}(q) = \{a_1, \ldots, a_m\}$ denote the set of acceptable gold answer strings for that question. Because benchmark answers may differ only in capitalization, punctuation, or articles, evaluation is performed after standard normalization. Let $\mathrm{norm}(\cdot)$ denote the normalization function that lowercases text and removes punctuation, articles, and extra whitespace [28], [60].

Some local generators emit an explicit leading `<think>...</think>` block before the final answer [49], [61]. To avoid scoring those internal traces as part of the response, evaluation strips any leading think block before lexical and semantic answer scoring.

The Exact Match (EM) score is a binary measure of whether the predicted answer exactly matches at least one normalized gold answer [60]:

$$\mathrm{EM}(q) = \max_{a \in \mathcal{A}(q)} \mathbf{1}[\mathrm{norm}(\hat{a}) = \mathrm{norm}(a)], \tag{11}$$

where $\mathbf{1}[\cdot]$ is the indicator function, which equals 1 when its condition is true and 0 otherwise. Thus, $\mathrm{EM}(q) = 1$ if the model prediction exactly matches at least one gold answer after normalization, and $\mathrm{EM}(q) = 0$ otherwise.

Because exact matching can be overly strict, we also report token-level F1, which gives partial credit when the predicted answer overlaps with a gold answer but is not an exact match. Let $\mathrm{tok}(s)$ denote the multiset of normalized tokens in a string s. For a predicted answer $\hat{a}$ and a gold answer a, let

$$c(\hat{a}, a) = |\mathrm{tok}(\mathrm{norm}(\hat{a})) \cap \mathrm{tok}(\mathrm{norm}(a))| \tag{12}$$

be the number of overlapping tokens, counting multiplicity. Using this overlap count, token precision and token recall are defined as

$$P(\hat{a}, a) = \frac{c(\hat{a}, a)}{|\mathrm{tok}(\mathrm{norm}(\hat{a}))|}, \quad R(\hat{a}, a) = \frac{c(\hat{a}, a)}{|\mathrm{tok}(\mathrm{norm}(a))|}. \tag{13}$$

The token-level F1 score for that gold answer is then

$$F_1(\hat{a}, a) = \begin{cases} \frac{2\,P(\hat{a},a)\,R(\hat{a},a)}{P(\hat{a},a)+R(\hat{a},a)}, & \text{if } P(\hat{a}, a) + R(\hat{a}, a) > 0, \\ 0, & \text{otherwise.} \end{cases} \tag{14}$$

Because a question may have multiple acceptable gold answers, the final token-level F1 score for question q is the maximum over the gold-answer set:

$$\mathrm{F1}(q) = \max_{a \in \mathcal{A}(q)} F_1(\hat{a}, a). \tag{15}$$

2) Retrieval metrics: Recall@k and R-precision.

To evaluate retrieval independently of answer generation, let $\mathcal{G}(q)$ denote the set of gold provenance passage identifiers associated with question q, and let $\mathcal{R}_k(q)$ denote the set of passage identifiers returned by the retriever in the top-k results for that question. Here, k is the retrieval depth (in this study, $k \in \{5, 10\}$).

The standard Recall@k metric measures what fraction of the gold provenance passages are present among the top-k retrieved passages:

$$\mathrm{Recall@}k(q) = \frac{|\mathcal{G}(q) \cap \mathcal{R}_k(q)|}{|\mathcal{G}(q)|}. \tag{16}$$

In this expression, $|\mathcal{G}(q) \cap \mathcal{R}_k(q)|$ is the number of gold provenance passages successfully retrieved in the top-k, and $|\mathcal{G}(q)|$ is the total number of gold provenance passages for that question. In the final analysis, this quantity is recomputed from the stored retrieved passage identifiers and gold provenance identifiers [62].

We also report R-precision, which evaluates precision at a cutoff equal to the number of gold provenance passages. Let

$$r(q) = |\mathcal{G}(q)| \tag{17}$$

be the number of gold provenance passages for question q. Then R-precision is defined as

$$\text{R-Prec}(q) = \frac{\left|\mathcal{G}(q) \cap \mathcal{R}_{r(q)}(q)\right|}{r(q)}. \tag{18}$$

This metric is useful because it adapts the cutoff to the number of relevant passages for each question rather than using a fixed k for all questions. As with Recall@k, the final analysis recomputes R-precision from the stored passage identifiers [62].

3) Provenance-constrained grounded metrics: EM and F1.

A system may produce a correct answer either because it genuinely used retrieved evidence or because the answer was already available in the generator's parametric memory. To distinguish these cases, we also compute grounded answer metrics that require retrieval success in addition to answer correctness [28], [63].

Let

$$\text{hit}(q) = \mathbf{1}[|\mathcal{G}(q) \cap \mathcal{R}_k(q)| \geq 1] \tag{19}$$

be an indicator that equals 1 when at least one gold provenance passage was retrieved in the top-k, and 0 otherwise. We also define

$$\text{all}(q) = \mathbf{1}[\mathcal{G}(q) \subseteq \mathcal{R}_k(q)]\,, \tag{20}$$

which equals 1 only when all gold provenance passages were retrieved.

Using $\text{hit}(q)$, the hit-based grounded versions of Exact Match and token-level F1 are

$$\text{gEM}_{\text{hit}}(q) = \text{EM}(q) \cdot \text{hit}(q), \quad \text{gF1}_{\text{hit}}(q) = \text{F1}(q) \cdot \text{hit}(q). \tag{21}$$

These quantities assign a nonzero score only when the model both answers correctly and retrieves at least one gold provenance passage.

Using $\text{all}(q)$, we also report stricter full-provenance grounded variants:

$$\text{gEM}_{\text{all}}(q) = \text{EM}(q) \cdot \text{all}(q), \quad \text{gF1}_{\text{all}}(q) = \text{F1}(q) \cdot \text{all}(q). \tag{22}$$

These stricter quantities assign a nonzero score only when the model answers correctly and retrieves the full gold provenance set. Taken together, the hit-based and all-based grounded metrics separate answer correctness from answer correctness supported by partially or fully recovered evidence.

4) Semantic answer similarity: SAS.

Lexical overlap does not capture all acceptable answers in local RAG. A generated answer may be semantically correct while differing in wording from the reference, especially when comparing different generator families and prompting behaviors. To complement EM and token-level F1, we therefore also compute semantic answer similarity (SAS) [29], [64].

Let $s_\psi(\hat{a}, a)$ denote a sentence-pair similarity model applied to the predicted answer $\hat{a}$ and a gold answer a. In the present implementation, s_ψ is a cross-encoder score produced by a sentence-transformers CrossEncoder model (default: `cross-encoder/stsb-roberta-large` [65]), normalized to the unit interval. The SAS score for question q is then

$$\text{SAS}(q) = \max_{a \in \mathcal{A}(q)} s_\psi(\hat{a}, a). \tag{23}$$

As with token-level F1, the maximum over the acceptable gold-answer set is used so that semantically equivalent alternative phrasings are not penalized unnecessarily.

To mirror the provenance-aware lexical analysis, we also define grounded semantic variants [28]:

$$\text{gSAS}_{\text{hit}}(q) = \text{SAS}(q) \cdot \text{hit}(q), \quad \text{gSAS}_{\text{all}}(q) = \text{SAS}(q) \cdot \text{all}(q). \tag{24}$$

These quantities preserve the distinction between semantic correctness alone and semantic correctness supported by partially or fully recovered gold provenance. SAS is computed only for completed answer-producing trials. Failed trials are retained in the robustness analysis rather than being assigned semantic quality scores.

5) Efficiency metrics.

In addition to quality metrics, we measure per-question runtime to characterize deployability under local hardware constraints. For each question q, we log the wall-clock time spent in query rewriting, retrieval, and answer generation. Let $T_{\text{rewrite}}(q)$ denote the time spent rewriting the query, $T_{\text{retrieve}}(q)$ the time spent retrieving passages, and $T_{\text{generate}}(q)$ the time spent producing the final answer. The total runtime for question q is

$$T_{\text{total}}(q) = T_{\text{rewrite}}(q) + T_{\text{retrieve}}(q) + T_{\text{generate}}(q). \tag{25}$$

For pipelines that do not include rewriting, $T_{\text{rewrite}}(q) = 0$. For the retrieval-only pipeline, $T_{\text{total}}(q) = T_{\text{retrieve}}(q)$.

In the aggregated analysis, we report the mean total runtime per generator and pipeline over all trials for each experimental configuration. This quantity is used in the quality—latency analysis to compare end-to-end runtime against answer quality across pipelines [66].

6) Robustness metrics.

In single-GPU local evaluation, a configuration can be operationally unattractive even if its average accuracy is high, because inference may stall, fail, or exceed practical latency budgets. We therefore report robustness metrics derived from per-trial execution outcomes [66], [67].

For a fixed experimental configuration c, let $\mathcal{D}_c$ denote the set of trials executed under that configuration. In this work, a configuration is defined by pipeline, generator, embedder, retrieval depth k, and context length; when multiple

knowledge-base variants are analyzed jointly, the knowledge-base tag is also treated as part of the configuration. For trial $i \in \mathcal{D}_c$, let $\text{status}_i \in \{\texttt{ok}, \texttt{failed}\}$ denote its execution status. Define the completion indicator

$$S_i = \mathbf{1}[\text{status}_i = \texttt{ok}]\,. \tag{26}$$

The completion rate and failure rate for configuration c are

$$\text{Comp}(c) = \frac{1}{|\mathcal{D}_c|} \sum_{i \in \mathcal{D}_c} S_i, \qquad \text{Fail}(c) = 1 - \text{Comp}(c). \tag{27}$$

To localize failures within the pipeline, let $\text{stage}_i \in \{\texttt{rewrite}, \texttt{retrieve}, \texttt{generate}\}$ denote the stage at which failed trial i terminated. The stage-specific failure rate for stage s is

$$\text{Fail}_s(c) = \frac{1}{|\mathcal{D}_c|} \sum_{i \in \mathcal{D}_c} \mathbf{1}[S_i = 0 \wedge \text{stage}_i = s]\,. \tag{28}$$

In the reported analysis, robustness is summarized primarily by failure rate across generators and pipelines, together with failure counts aggregated by pipeline stage. These summaries are computed over all trials for each configuration, and failed trials are retained explicitly rather than discarded. This design preserves operational failures in the final aggregates and avoids biasing the robustness analysis toward only successful executions.

The runner writes one record per trial, including execution status, failure stage, total runtime, retrieved passage identifiers, and gold provenance identifiers. Failed trials are written explicitly rather than discarded. The raw results therefore support post hoc computation of standard Recall@k, R-precision, grounded hit-based and all-based lexical metrics, grounded semantic metrics, and the robustness metrics above.

During streamed model generation in the rewriting and answering stages, the serving layer enforces two watchdog thresholds: a time-to-first-token threshold and a total stream-runtime threshold. If no token is emitted before the first threshold expires, the trial fails with a no-first-token timeout; if total stream time exceeds the second threshold, the trial fails with a stream timeout. These failures are included in the aggregated results rather than being omitted. Timeout thresholds were set to 300 seconds for time-to-first-token and 600 seconds for complete stream timeout.

V. EXPERIMENTAL RESULTS

This section reports the main empirical results for the shared retrieval-only condition (P0), closed-book answering (P1), standard retrieve-then-answer RAG (P2), and same-model rewrite-RAG (P3).

A. RETRIEVAL PERFORMANCE AND RETRIEVAL COST

The shared retrieval condition (P0) already separates the three embedders clearly. QwenEmb-4B is the strongest retriever in provenance terms, reaching 96.8% Recall@5 and 98.0% Recall@10, together with 68.5% R-Precision. MiniLM-L6 is close behind at 94.4% Recall@5 and 96.6% Recall@10, with 67.1–67.2% R-Precision. Nomic-v1.5 is substantially weaker, reaching 60.0% Recall@5 and 65.2% Recall@10 with about 30.6% R-Precision.

The retrieval-time differences are much larger than the recall differences between the two stronger embedders. MiniLM-L6 retrieves in about 0.015 seconds per query, whereas QwenEmb-4B takes about 2.28 seconds. Although QwenEmb-4B improves average Recall and R-Precision by only about 2% over MiniLM-L6, MiniLM-L6 is roughly 150× faster, or about two orders of magnitude faster. In other words, the strongest retriever and the fastest retriever are not the same system. This gap is carried forward into the generator results, where the retriever choice remains relevant not only for provenance recovery but also for end-to-end deployability.

B. END-TO-END LEXICAL QA RESULTS

Across all 12 generators, retrieval-augmented answering improves substantially over closed-book generation, as shown in Tables 4 and 5. The strongest overall lexical result is obtained by Qwen3:30B, whose best P2 configuration (QwenEmb-4B and $k = 10$) reaches 58.4% F1 and 46.0% EM. The next strongest P2 points are Qwen3:4B at 51.0% F1 and 38.8% EM, Gemma3:12B reaches 46.6% best P2 F1 and 28.6% best P2 EM, Gemma3:4B at 42.9% F1 and 24.4% EM, and GPT-OSS:20B at 39.6% F1 and 22.0% EM. Even for the smallest Qwen model, retrieval changes the operating point substantially: Qwen3:0.6B rises from 7.5% F1 in P1 to 22.1% in its best P2 setting.

At the same time, the lexical results are not monotonic in nominal parameter count, as shown by Figure 1. Within the Qwen ladder, Qwen3:4B is clearly stronger than Qwen3:8B and Qwen3:14B in the present local setup, and Qwen3:1.7B also exceeds both of those larger checkpoints in best P2 F1. The cross-family comparison points show the same general pattern: Gemma3:12B and Gemma3:4B both rank above several larger Qwen checkpoints, while GPT-OSS:20B is competitive with the stronger mid-range models without being the top lexical model overall.

Figure 2 shows that the best P2 configuration is usually, but not always, obtained with QwenEmb-4B. It provides the strongest P2 settings for most of the higher-scoring generators, including Qwen3:30B, Qwen3:4B, Gemma3:12B, Gemma3:4B, and Qwen3:1.7B. However, MiniLM-L6 pro-

TABLE 3. Retrieval-only performance (P0) on the reduced KILT-NQ knowledge base, aggregated across the shared retrieval setting. Recall@k and R-Precision are reported for each embedding model at top-$k = 5$ and top-$k = 10$, together with mean retrieval time per query.

	Recall@k (%)		R-Prec (%)		Mean retrieval time (s)	
Embedder	$k = 5$	$k = 10$	$k = 5$	$k = 10$	$k = 5$	$k = 10$
MiniLM-L6	94.4	96.6	67.1	67.2	0.015	0.015
Nomic-v1.5	60.0	65.2	30.6	30.6	2.097	2.097
QwenEmb-4B	96.8	98.0	68.5	68.5	2.285	2.282

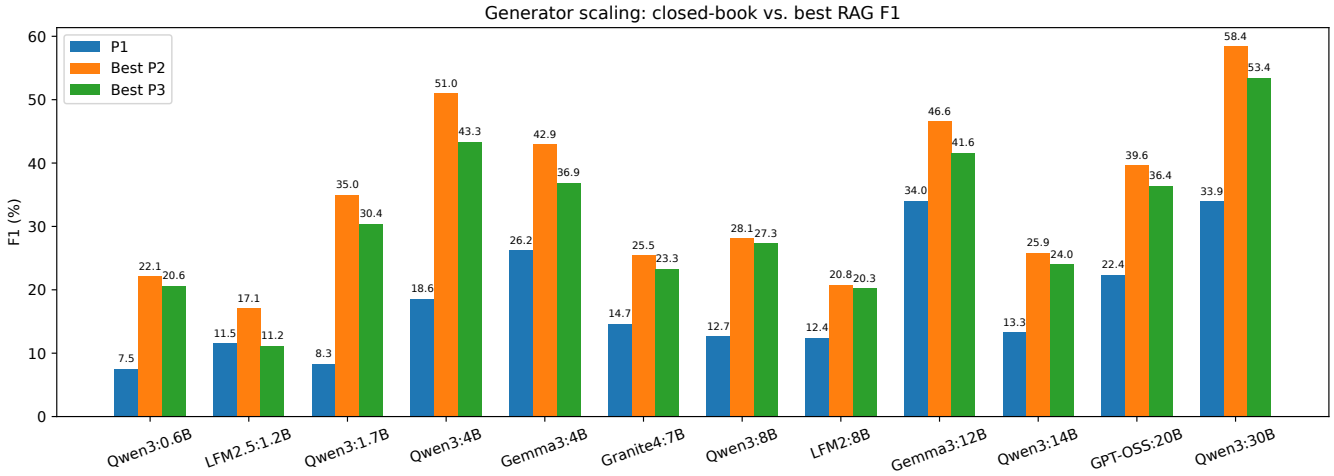

FIGURE 1. Generator scaling: closed-book vs. best RAG F1. For each generator, the plotted P2 and P3 values are the best F1 scores obtained across the evaluated embedde

TABLE 4. Lexical answer quality by generator and pipeline. P1 reports closed-book F1. For P2 and P3, each cell reports the best F1 obtained for that generator at the given top-k across the evaluated embedders.

		Best P2 F1 (%)		Best P3 F1 (%)	
Generator	**P1 F1 (%)**	$k = 5$	$k = 10$	$k = 5$	$k = 10$
Qwen3:0.6B	7.5	22.1	19.8	19.8	20.6
LFM2.5:1.2B	11.5	13.0	17.1	9.5	11.2
Qwen3:1.7B	8.3	34.0	35.0	30.4	29.6
Qwen3:4B	18.6	51.0	48.0	43.3	43.2
Gemma3:4B	26.2	42.9	40.2	36.9	36.0
Granite4:7B	14.7	25.5	24.0	23.3	21.2
Qwen3:8B	12.7	27.9	28.1	26.7	27.3
LFM2:8B	12.4	20.8	20.0	20.3	19.4
Gemma3:12B	34.0	46.4	46.6	41.5	41.6
Qwen3:14B	13.3	25.5	25.9	24.0	24.0
GPT-OSS:20B	22.4	39.6	38.6	36.1	36.4
Qwen3:30B	33.9	56.4	58.4	52.4	53.4

TABLE 5. Lexical answer quality by generator and pipeline. P1 reports closed-book EM. For P2 and P3, each cell reports the best EM obtained for that generator at the given top-k across the evaluated embedders.

		Best P2 EM (%)		Best P3 EM (%)	
Generator	**P1 EM (%)**	$k = 5$	$k = 10$	$k = 5$	$k = 10$
Qwen3:0.6B	0.2	8.2	6.6	6.8	6.8
LFM2.5:1.2B	1.8	7.2	10.6	4.0	6.0
Qwen3:1.7B	0.2	15.4	16.6	12.0	15.0
Qwen3:4B	6.8	38.8	36.6	31.8	31.2
Gemma3:4B	13.0	24.4	21.0	21.2	18.6
Granite4:7B	0.8	8.8	7.0	7.0	6.4
Qwen3:8B	0.6	7.4	7.4	7.8	7.4
LFM2:8B	0.6	1.6	1.4	2.0	1.6
Gemma3:12B	19.0	28.6	27.6	23.4	21.2
Qwen3:14B	1.4	5.4	4.8	5.6	4.2
GPT-OSS:20B	8.8	22.0	20.6	18.8	18.6
Qwen3:30B	18.0	44.8	46.0	41.8	43.0

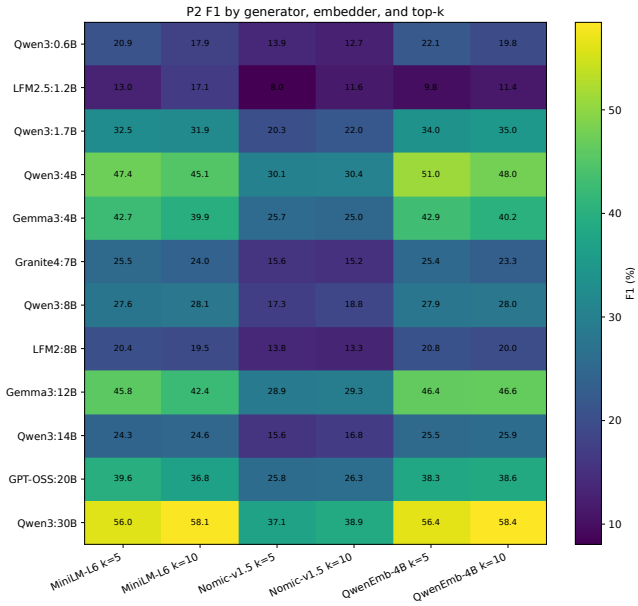

FIGURE 2. P2 F1 (%) by generator, embedding model, and retrieval depth ($k \in \{5, 10\}$).

vides the best P2 setting for GPT-OSS:20B, Granite4:7B, LFM2.5:1.2B, and Qwen3:8B. Nomic-v1.5 rarely yields the strongest end-to-end configuration in the current setup.

The rewrite-RAG condition of P3 (Figure 3) remains weaker than standard P2 for every generator when compared by best overall configuration. The gap is large for several of the stronger models: Qwen3:4B drops from 51.0% to 43.3% F1, Gemma3:12B from 46.6% to 41.6%, GPT-OSS:20B from 39.6% to 36.4%, and Qwen3:30B from 58.4% to 53.4%. Smaller models show the same overall pattern, although the gap is sometimes smaller. In the present experiment, P3 does

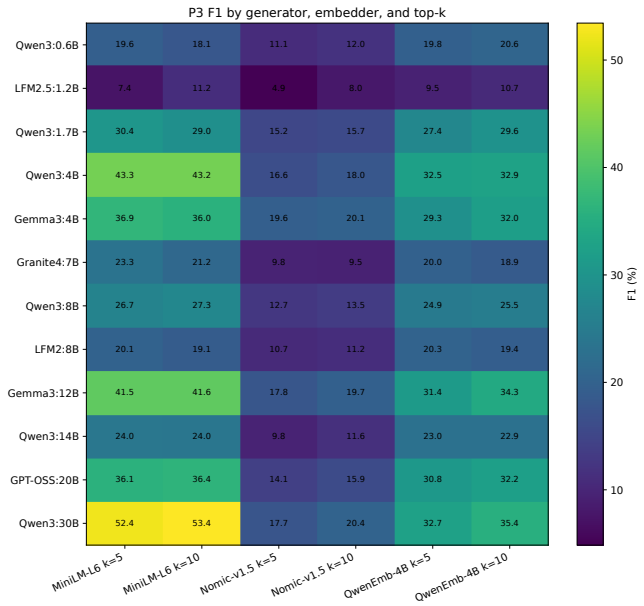

FIGURE 3. P3 F1 (%) by generator, embedding model, and retrieval depth ($k \in \{5, 10\}$).

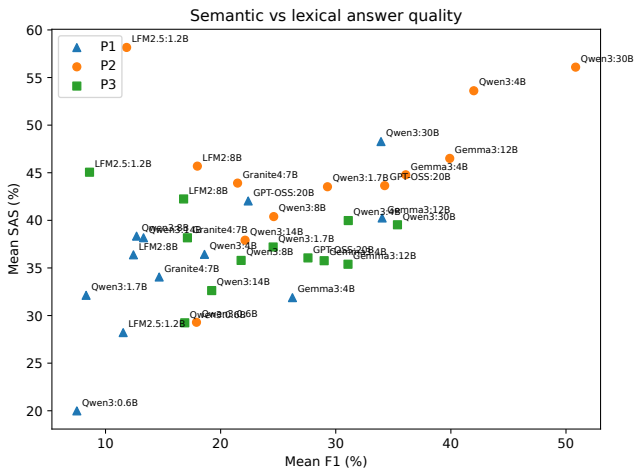

FIGURE 4. Semantic vs. lexical answer quality across P1, P2, and P3. Each point corresponds to one generator–pipeline pair, with mean F1 on the x-axis and mean SAS on the y-axis.

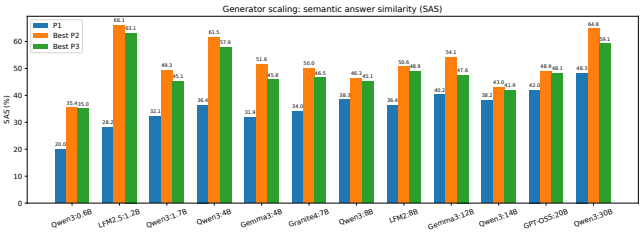

FIGURE 5. Generator scaling: closed-book vs. best RAG semantic answer similarity (SAS). For each generator, the plotted P2 and P3 values are the best SAS scores obtained across the evaluated embedders and top-k settings.

not overtake P2 as the strongest lexical pipeline for any generator.

C. GROUNDED AND SEMANTIC EVALUATION

As shown in Table 6, the stricter full-provenance grounded evaluation preserves the main ranking trends while reducing absolute scores, as expected. Under P2, Qwen3:30B remains the best-performing model, achieving 40.8% gF1-all and 32.2% gEM-all at $k = 10$. Qwen3:4B ranks next with 33.0% gF1-all and 25.6% gEM-all, followed closely by Gemma3:12B with 32.3% gF1-all and 19.0% gEM-all. GPT-OSS:20B achieves 26.0% gF1-all and 13.2% gEM-all in its best setting under the stricter grounded criterion. Consistent with the lexical results, P3 performs worse than P2 for all generators under this stricter evaluation.

Semantic evaluation adds another view of answer quality. Figure 5 shows the corresponding semantic scaling pattern across generators and pipelines. The strongest lexical models also remain strong semantically: Qwen3:30B reaches 64.8% best P2 SAS and Qwen3:4B reaches 61.5%. Gemma3:12B and Gemma3:4B also remain strong under SAS, reaching 54.1% and 51.6%, respectively. As in the lexical scaling results, P2 remains stronger than P3 for every generator in best overall semantic performance.

Figure 4 adds a different perspective by comparing semantic and lexical quality directly. It shows that the lexical and semantic rankings do not coincide perfectly. Several generators appear substantially stronger under semantic similarity than under lexical overlap alone. LFM2.5:1.2B reaches 66.1% best P2 SAS despite only 17.1% best P2 F1, LFM2:8B reaches 50.6% SAS at 20.8% F1, and Granite4:7B reaches 50.0% SAS at 25.5% F1.

Taken together, the grounded (Table 6) and semantic (Figures 5 and 4) results show two distinct patterns. First, the strongest lexical models retain their advantage when the answer must also be supported by fully recovered provenance. Second, semantic similarity widens the picture for several smaller and cross-family generators whose answers appear closer to the reference meaning than their lexical overlap alone would suggest.

D. QUALITY–LATENCY TRADE-OFF AND RUNTIME ROBUSTNESS

Figure 6 summarizes the quality–latency frontier using the best configuration of each pipeline for each generator. Among the best P2 settings, mean total time per question ranges from 2.74 seconds for Granite4:7B up to 18.15 seconds for Qwen3:30B. Several models cluster in a mid-latency region around 4.8–5.4 seconds per question, including LFM2:8B, Gemma3:4B, Gemma3:12B, GPT-OSS:20B, and LFM2.5:1.2B. Qwen3:4B provides a stronger but slower point at 15.56 seconds, while Qwen3:30B occupies the highest-quality but also one of the slowest P2 points. The best P3 settings are systematically slower, ranging from 9.35

TABLE 6. Combined grounded-answer results for P2 and P3 using the stricter *all*-based grounding criterion, which requires retrieval of the full gold provenance set. The table reports gEM-all and gF1-all for each generator at retrieval depths $k = 5$ and $k = 10$.

	P2 gEM-all (%)		P3 gEM-all (%)		P2 gF1-all (%)		P3 gF1-all (%)	
Generator	$k = 5$	$k = 10$	$k = 5$	$k = 10$	$k = 5$	$k = 10$	$k = 5$	$k = 10$
Qwen3:0.6B	4.2	5.0	4.0	4.4	13.3	14.4	11.3	14.0
LFM2.5:1.2B	3.8	6.8	2.4	3.8	6.8	10.8	5.8	6.8
Qwen3:1.7B	8.8	10.6	7.0	8.6	19.7	23.6	16.4	17.1
Qwen3:4B	22.6	25.6	16.0	19.2	29.9	33.0	21.7	25.0
Gemma3:4B	13.8	14.2	12.4	11.6	25.0	27.9	20.7	21.6
Granite4:7B	4.4	3.4	3.6	3.0	14.2	14.8	12.2	12.5
Qwen3:8B	4.0	3.8	3.2	4.2	16.6	18.5	14.3	17.1
LFM2:8B	0.6	0.6	0.4	0.8	12.1	13.1	10.7	12.1
Gemma3:12B	17.6	19.0	13.2	12.4	27.7	32.3	21.9	23.9
Qwen3:14B	2.6	2.4	2.6	2.4	15.0	17.3	14.2	15.7
GPT-OSS:20B	11.2	13.2	9.2	9.6	21.3	26.0	18.5	20.6
Qwen3:30B	27.8	32.2	22.6	26.2	34.6	40.8	27.6	31.7

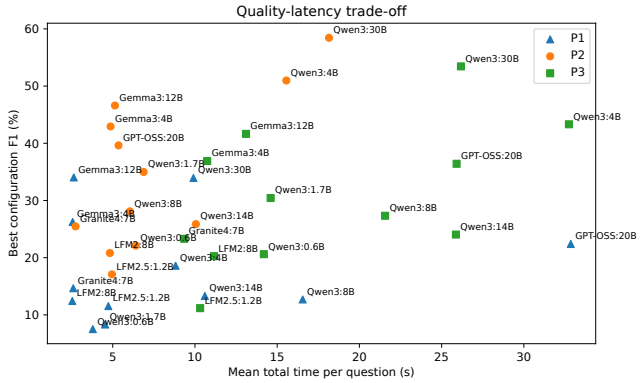

FIGURE 6. Quality–latency trade-off per pipeline. The x-axis shows the mean end-to-end time per question (s), and the y-axis shows the corresponding best F1 (%).

seconds to 32.74 seconds per question.

Robustness follows the same broad separation between standard RAG and rewrite-RAG. P0 had no failures. Across answer-producing pipelines, the run recorded 242 failed trials in total: 51 in P1 generation, 51 in P2 generation, 82 in P3 rewriting, and 58 in P3 generation. Thus, P3 accounts for the largest share of failures, and its failures are split across both rewriting and answering stages.

At the generator level, P2 remains relatively stable. The

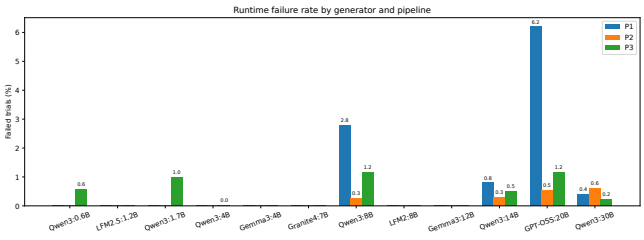

FIGURE 7. Runtime failure rate (%) by generator and pipeline.

TABLE 7. Runtime failure count per pipeline and stage.

Pipeline	Error stage	Count
P3	rewrite	82
P3	generate	58
P2	generate	51
P1	generate	51

largest P2 failure rates are 0.60% for Qwen3:30B, 0.53% for GPT-OSS:20B, and 0.30% for Qwen3:14B, while several generators complete all P2 trials without recorded failures. P3 shows higher failure rates overall, reaching 1.17% for GPT-OSS:20B and Qwen3:8B, 1.00% for Qwen3:1.7B, and 0.57% for Qwen3:0.6B. Closed-book P1 is mostly stable, although GPT-OSS:20B and Qwen3:8B stand out with 6.2% and 2.8% failure rates, respectively.

Taken together, the quality–latency and robustness results identify the practical operating envelope observed on this workstation. In the present setting, P2 provides the strongest overall balance of quality, latency, and stability. P3 increases total runtime and failure exposure without surpassing the best P2 results in lexical or stricter grounded evaluation.

VI. DISCUSSION

This section interprets the main findings of our study.The results support a broader point than a simple ranking of models or pipelines. Under a fixed single-GPU budget, local RAG is not merely a smaller version of cloud or server-side RAG. It is a distinct evaluation regime in which retrieval quality, lexical answer quality, semantic adequacy, latency, and runtime robustness do not collapse into a single ordering. In the present study, the strongest retriever is not the fastest retriever, the largest generator is not always the strongest practical deployment point, query rewriting is not automatically beneficial, and semantic similarity does not always track lexical overlap perfectly. Taken together, these findings reinforce the central motivation of the paper: controlled workstation-scale experiments remain scientifically valuable

precisely because they expose trade-offs that are easy to overlook in cloud-first evaluation.

A. ANSWERING THE RESEARCH QUESTIONS

The experimental results give clear answers to the four research questions.

RQ1 — Retriever impact. Embedding choice matters strongly, but not only through retrieval quality. QwenEmb-4B is the strongest retriever in provenance terms and supports many of the strongest P2 configurations. However, MiniLM-L6 remains close in retrieval quality while being about two orders of magnitude faster, and in several cases it supports the best end-to-end P2 configuration. Nomic-v1.5 is clearly weaker in the present setup. The main implication is that retriever evaluation in local RAG should not be framed only as a recall problem. Under local constraints, retrieval quality and retrieval cost must be evaluated jointly.

RQ2 — Generator impact. Generator choice matters substantially, but the scaling pattern is not monotonic in nominal parameter count. Qwen3:30B is the strongest overall lexical model in the current study, yet Qwen3:4B emerges as a much stronger deployment point than Qwen3:8B and Qwen3:14B in the present quantized local setup. The broader comparison models strengthen this conclusion: Gemma3:12B and Gemma3:4B outperform several larger Qwen checkpoints, and GPT-OSS:20B is competitive without becoming the dominant model overall. This means that local generator selection cannot be inferred from model size alone. Under a fixed 24 GB VRAM budget, end-to-end performance depends on a mixture of family-specific behavior, quantization format, prompt sensitivity, retrieved-context use, and runtime characteristics.

RQ3 — Single-model query rewriting. Same-model query rewriting is not worthwhile in the present setup. Across the full generator set, the best P3 configurations do not surpass the best P2 configurations in lexical, stricter grounded, or semantic performance. At the same time, P3 is slower and less robust because it introduces an additional generation stage. This makes P3 an informative negative result. Query rewriting may be useful in other RAG settings, but in this local setting the added rewrite step increases system complexity without providing enough end-to-end benefit to justify its cost.

RQ4 — Robustness under local constraints. Runtime failures are not incidental enough to ignore in local RAG evaluation. In the present study, failures are concentrated in generation-bearing stages and are especially associated with the rewrite-based pipeline. P0 has no failures, P2 remains comparatively stable, and P3 carries the largest failure burden because it adds both a rewriting stage and an answering stage. This confirms that robustness should be treated as part of model quality in local deployment rather than as separate engineering noise.

B. WHAT THE RESULTS MEAN FOR LOCAL RAG

A central result of this paper is that single-workstation evaluation reveals trade-offs that are easy to hide in larger infrastructures. In a server-side or cloud setting, it is tempting to treat retrieval as a background service, generation as the primary variable of interest, and occasional runtime instability as an implementation detail. The present results show that this view is too narrow for local RAG. On limited hardware, a small change in retrieval quality may come with a large change in retrieval time; a model with more parameters may not produce a better end-to-end result; an extra generation step may harm both latency and stability; and a system that looks acceptable under one metric may look meaningfully different under another.

This is why the paper argues for multi-axis evaluation. If the analysis were limited to lexical answer quality alone, several important aspects of the system would be missed. The retrieval results show that provenance recovery and retrieval cost can point in different directions. The stricter grounded scores show that answer quality and evidence-grounded answer quality are not the same. The latency and failure summaries show that practical deployability is not reducible to best-case F1. In this sense, local RAG should be evaluated as a system rather than as a generator-only benchmark.

The semantic results strengthen this point further. Overall, the results suggest a positive relationship between lexical quality and semantic quality: the strongest F1 models also tend to remain strong under SAS. This is reassuring, because it indicates that the lexical and semantic evaluations are not describing unrelated phenomena. However, the outliers are scientifically important. The most striking case is LFM2.5:1.2B, which achieves the strongest best-P2 SAS in the current study despite relatively weak lexical F1. It also exceeds its larger relative, LFM2:8B, on semantic quality. This suggests that some smaller or differently optimized models may preserve answer meaning better than lexical overlap alone implies. At the same time, the grounded and robustness results show why SAS should not replace the other metrics. A model that is semantically promising but weak in lexical exactness, provenance-grounding, or operational stability is not automatically the best deployment choice. The more defensible conclusion is that semantic evaluation usefully broadens the picture, especially in heterogeneous local model comparisons, and helps identify models whose answer behavior would otherwise be underestimated by EM and F1 alone.

This is particularly relevant to the broader argument of the paper. Much of the current literature on efficient, private, or edge-oriented RAG still emphasizes architecture, serving, or application-specific outcomes, often under infrastructure assumptions that differ from workstation-scale deployment. Our results show that once the system is constrained to a single consumer GPU, the relevant question is not simply which model is best in theory, but which configuration remains accurate enough, grounded enough, fast enough, and

stable enough to be used in practice. That is exactly the kind of question that controlled local experiments are well suited to answer.

C. WHY SINGLE-WORKSTATION STUDIES MATTER

One of the motivations of this work was the relative lack of controlled experiments for local RAG systems that do not depend on clusters, servers, or cloud services. The present results strengthen that motivation. They show that workstation-scale evaluation is not merely a reduced or pedagogical version of larger-scale benchmarking. Instead, it highlights a different set of decisions: whether a compact retriever is good enough, whether a mid-scale generator is a better operating point than a larger one, whether query rewriting is worth the extra stage, and whether a model that looks good in average accuracy is actually practical once latency and failures are counted.

This point matters for both research and practice. For researchers, single-workstation experiments provide a reproducible setting in which component trade-offs can be studied under explicit constraints rather than abstracted away by abundant infrastructure. For practitioners, especially those building private, low-cost, or on-device systems, these trade-offs determine which configurations are actually usable. In that sense, the contribution of this paper is not only the performance comparison itself, but also the argument that local RAG deserves evaluation protocols tailored to its real deployment conditions.

D. STACK, REPRODUCIBILITY, AND SCOPE

At the same time, the present results should be interpreted as results for this stack. The precise ranking of models and pipelines may change under a different local runtime, different quantized variants, different prompts, a different GPU, a different timeout policy, or a different retrieval stack, including different model-serving frameworks beyond Ollama and different retrieval/indexing frameworks beyond FAISS. This is especially important for the non-monotonic scaling patterns observed here. The main lesson is therefore not that one specific model family or checkpoint is universally superior, but that workstation-scale local deployment must be tested empirically rather than inferred from nominal model size or abstract benchmark reputation.

For this reason, the code, processed results, tables, and figures for this project will be made available so that others can reproduce the study in their own local stacks and extend it with additional models, retrievers, or serving environments. This is part of the scientific contribution. If local RAG remains under-studied relative to cloud-based evaluation, then reproducible workstation-scale pipelines are useful not only for the conclusions they produce directly, but also as baselines that others can rerun, stress-test, and build upon.

The study also has clear methodological limits. The QA evaluation uses a 500-question subset rather than the full benchmark [28]. The corpus is reduced rather than a full Wikipedia-scale index. The retrieval stack is intentionally simple, with no reranking, no MMR-style diversification, and a fixed context length [68], [69]. The cross-family model comparison is targeted rather than exhaustive, and the system is evaluated through one local serving stack rather than several. For these reasons, the paper should not be read as a universal ranking of all local RAG systems. Its contribution is narrower but more practical: it characterizes how quality, grounding, semantic adequacy, latency, and robustness interact under a realistic single-workstation constraint.

Taken together, these limitations do not weaken the central argument. Rather, they define its scope. The paper does not claim that all local RAG deployments will reproduce the same ordering. It claims that once deployment is constrained to a single workstation, those orderings become empirical questions that must be evaluated jointly across quality and systems dimensions. The present results support that claim.

VII. CONCLUSION

This paper examined retrieval-augmented generation under a setting that is common in practice but still underrepresented in the literature: a single local workstation running quantized models without cloud, cluster, or server infrastructure. Rather than proposing a new retriever or generator architecture, the paper studied how a local RAG system behaves when retrieval quality, lexical answer quality, semantic adequacy, latency, and runtime robustness are evaluated together under explicit deployment constraints.

Several conclusions follow. Retrieval-augmented answering substantially improves over closed-book generation across the full generator set, but retriever choice matters not only for provenance recovery but also for end-to-end deployability. The strongest retriever is not the fastest one, and compact retrieval can remain highly competitive once latency is considered. Generator performance under local quantized execution is also not monotonic in nominal model size. Although Qwen3:30B achieves the strongest overall lexical performance, models such as Qwen3:4B and Gemma3:12B emerge as strong practical alternatives, showing that model selection under local constraints cannot be inferred from parameter count alone. Same-model query rewriting is not justified in the present setup, because it increases latency and failure exposure without surpassing standard RAG in lexical, grounded, or semantic performance. Semantic evaluation also adds useful information beyond EM and F1 as stronger lexical models generally remain strong under SAS, but informative outliers, especially LFM2.5:1.2B, show that lexical overlap alone can understate some models' answer quality.

Taken together, these findings support the paper's broader claim that local RAG is not merely a smaller version of cloud-scale benchmarking, but a distinct evaluation setting. In this setting, retrieval quality, grounding, semantic adequacy, latency, and robustness must all be treated as primary outcomes. At the same time, the rankings reported here are specific to the evaluated stack, including the serving and retrieval frameworks, quantized model variants, prompts, hardware,

and timeout configuration. To support reproducibility, the code, processed results, and derived artifacts from this project will be released so that others can replicate the study in their own local environments and extend it further. Evaluation on a single workstation is not a substitute for larger-scale experiments, but it should be regarded as a scientifically useful and necessary complement to them.

Taken together, these findings support the broader argument of the paper that local RAG is not simply a reduced version of cloud-scale benchmarking, but a distinct evaluation regime in which retrieval, grounding, semantic adequacy, latency, and robustness must all be treated as primary outcomes. At the same time, the precise rankings reported here remain specific to this stack, including the chosen serving and retrieval frameworks, quantized variants, prompts, hardware, and timeout settings. For that reason, the code, processed results, and derived artifacts from this project will be made available so that others can reproduce the study in their own local stacks and build on it. Single-workstation evaluation will not replace larger-scale experiments, but it should be treated as a necessary complement to them.

REFERENCES

[1] P. Lewis et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," in Advances in Neural Information Processing Systems, 2020.
[2] J. Chen et al., "Benchmarking Large Language Models in Retrieval-Augmented Generation," AAAI 2024, 2024.
[3] Z. Shi et al., "Generate-then-Ground in Retrieval-Augmented Generation for Multi-hop Question Answering," in Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics, 2024.
[4] A. Asai et al., "SELF-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection," in International Conference on Learning Representations, 2024.
[5] K. Guu et al., "REALM: Retrieval-Augmented Language Model Pre-Training," in International Conference on Machine Learning, 2020.
[6] J. Lin et al., "AWQ: Activation-Aware Weight Quantization for On-Device LLM Compression and Acceleration," in Proceedings of the 7th MLSys Conference, 2024.
[7] D. Rau et al., "BERGEN: A Benchmarking Library for Retrieval-Augmented Generation," 2024, arXiv:2407.01102.
[8] A. Gan, H. Yu, K. Zhang, Q. Liu, W. Yan, Z. Huang, S. Tong, and G. Hu, "Retrieval Augmented Generation Evaluation in the Era of Large Language Models: A Comprehensive Survey," 2025, arXiv:2504.14891.
[9] J. Li, C. Xu, L. Jia, F. Wang, C. Zhang, and J. Liu, "EACO-RAG: Towards Distributed Tiered LLM Deployment using Edge-Assisted and Collaborative RAG with Adaptive Knowledge Update," 2024, arXiv:2410.20299.
[10] S. Liu, Z. Zheng, X. Huang, F. Wu, G. Chen, and J. Wu, "Efficient Distributed Retrieval-Augmented Generation for Enhancing Language Model Performance," 2025, arXiv:2504.11197.
[11] W. Zhou, Y. Yan, and Q. Yang, "DGRAG: Distributed Graph-based Retrieval-Augmented Generation in Edge-Cloud Systems," 2025, arXiv:2505.19847.
[12] W. Sun, J. Wang, Q. Guo, Z. Li, W. Wang, and R. Hai, "CEBench: A Benchmarking Toolkit for the Cost-Effectiveness of LLM Pipelines," 2024, arXiv:2407.12797.
[13] K. Vrettos and M. E. Klontzas, "Accurate and Energy Efficient: Local Retrieval-Augmented Generation Models Outperform Commercial Large Language Models in Medical Tasks," 2025, arXiv:2506.20009.
[14] I.-T. Sorodoc et al., "GaRAGe: A Benchmark with Grounding Annotations for RAG Evaluation," in Findings of the Association for Computational Linguistics: ACL 2025, 2025.
[15] A. Radford et al., "Language Models are Unsupervised Multitask Learners," 2019, openAI technical report.
[16] C. Raffel et al., "Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer," Journal of Machine Learning Research, 2020.
[17] F. Petroni et al., "Language Models as Knowledge Bases?" in Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing, 2019.
[18] N. Kandpal et al., "Large Language Models Struggle to Learn Long-Tail Knowledge," in International Conference on Machine Learning, 2023.
[19] N. D. Cao et al., "Editing Factual Knowledge in Language Models," in Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing, 2021.
[20] V. Karpukhin et al., "Dense Passage Retrieval for Open-Domain Question Answering," in Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing, 2020.
[21] J. Johnson et al., "Billion-scale Similarity Search with GPUs," in IEEE Transactions / arXiv preprint 1702.08734, 2017.
[22] Y. Gao et al., "Retrieval-Augmented Generation for Large Language Models: A Survey," 2024, arXiv:2312.10997.
[23] X. Ma et al., "Query Rewriting for Retrieval-Augmented Large Language Models," in Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing, 2023.
[24] E. Frantar et al., "GPTQ: Accurate Post-Training Quantization for Generative Pre-Trained Transformers," in International Conference on Learning Representations, 2023.
[25] G. Xiao et al., "SmoothQuant: Accurate and Efficient Post-Training Quantization for Large Language Models," in International Conference on Machine Learning, 2023.
[26] U. Kurt, "Which Quantization Should I Use? A Unified Evaluation of llama.cpp Quantization on Llama-3.1-8B-Instruct," 2026, arXiv:2601.14277.
[27] L. Sparrenberg et al., "Small and Fast LLMs on Commodity Hardware: Post-Training Quantization in llama.cpp," IEEE DSAA 2025, 2025.
[28] F. Petroni et al., "KILT: a Benchmark for Knowledge Intensive Language Tasks," in Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, 2021.
[29] H. Yu, A. Gan, K. Zhang, S. Tong, Q. Liu, and Z. Liu, "Evaluation of Retrieval-Augmented Generation: A Survey," 2024, arXiv:2405.07437.
[30] W. Kwon et al., "Efficient Memory Management for Large Language Model Serving with PagedAttention," in Proceedings of the 29th Symposium on Operating Systems Principles, 2023.
[31] P. P. Ray and M. P. Pradhan, "An Evaluation Framework for Measuring Prompt-wise Metrics for Large Language Models in Resource-Constrained Edge," BenchCouncil Transactions on Benchmarks, Standards and Evaluations, 2025.
[32] T. T. Nguyen and T. Nguyen, "An Evaluation of LLMs Inference on Popular Single-board Computers," 2025, arXiv:2511.07425.
[33] G. Izacard and E. Grave, "Leveraging Passage Retrieval with Generative Models for Open Domain Question Answering," in Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics, 2021.
[34] L. Gao et al., "Precise Zero-Shot Dense Retrieval without Relevance Labels," 2022, arXiv:2212.10496.
[35] Y. Mao et al., "Generation-Augmented Retrieval for Open-Domain Question Answering," in Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing, 2021.
[36] N. Thakur et al., "BEIR: A Heterogeneous Benchmark for Zero-shot Evaluation of Information Retrieval Models," in Advances in Neural Information Processing Systems, 2021.
[37] N. Muennighoff et al., "MTEB: Massive Text Embedding Benchmark," in Proceedings of the 17th Conference of the European Chapter of the Association for Computational Linguistics, 2023.
[38] S. Barnett et al., "Seven Failure Points When Engineering a Retrieval Augmented Generation System," in Proceedings of the 3rd International Conference on AI Engineering — Software Engineering for AI, 2024.
[39] J. Saad-Falcon et al., "ARES: An Automated Evaluation Framework for Retrieval-Augmented Generation Systems," 2024, arXiv:2311.09476.
[40] D. Ru et al., "RAGCHECKER: A Fine-grained Framework for Diagnosing Retrieval-Augmented Generation," 2024, arXiv:2408.08067.
[41] S. Muller et al., "GroUSE: A Benchmark to Evaluate Evaluators in Grounded Question Answering," in Proceedings of the 31st International Conference on Computational Linguistics, 2025.
[42] T. Dettmers et al., "LLM.int8(): 8-bit Matrix Multiplication for Transformers at Scale," in Advances in Neural Information Processing Systems, 2022.

[43] Y. Song et al., "PowerInfer: Fast Large Language Model Serving with a Consumer-grade GPU," in Proceedings of the 30th Symposium on Operating Systems Principles, 2024.
[44] Z. J. Wang and D. H. Chau, "MeMemo: On-device Retrieval Augmentation for Private and Personalized Text Generation," in Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval, 2024.
[45] K. Seemakhupt et al., "EdgeRAG: Online-Indexed RAG for Edge Devices," 2024, arXiv:2412.21023.
[46] W. Yu et al., "RAGDoll: Efficient Offloading-based Online RAG System on a Single GPU," 2025, arXiv:2504.15302.
[47] A. Wada et al., "Retrieval-Augmented Generation Elevates Local LLM Quality in Radiology Contrast Media Consultation," npj Digital Medicine, 2025.
[48] K. Enevoldsen et al., "MMTEB: Massive Multilingual Text Embedding Benchmark," in International Conference on Learning Representations, 2025.
[49] Qwen Team, "Qwen3 Technical Report," 2025, arXiv:2505.09388.
[50] A. Amini, A. Banaszak, H. Benoit, A. Böök, T. Dakhran, S. Duong, A. Eng, F. Fernandes, M. Härkönen, A. Harrington, R. Hasani, S. Karwa, Y. Khrustalev, M. Labonne, M. Lechner, V. Lechner, S. Lee, Z. Li, N. Loo, J. Marks, E. Mosca, S. J. Paech, P. Pak, R. N. Parnichkun, A. Quach, R. Rogers, D. Rus, N. Saxena, B. Schlager, T. Seyde, J. T. H. Smith, A. Tadimeti, and N. Tumma, "Lfm2 technical report," arXiv preprint arXiv:2511.23404, 2025.
[51] Liquid AI, "Lfm2.5-1.2b-thinking: On-device reasoning under 1gb," https://www.liquid.ai/blog/lfm2-5-1-2b-thinking-on-device-reasoning-under-1gb, 2026, official release post.
[52] Gemma Team, "Gemma 3 technical report," arXiv preprint arXiv:2503.19786, 2025.
[53] IBM, "Ibm granite 4.0: Hyper-efficient, high performance hybrid models," https://www.ibm.com/new/announcements/ibm-granite-4-0-hyper-efficient-high-performance-hybrid-models, 2025, official announcement for the Granite 4 family.
[54] S. Agarwal et al., "gpt-oss-120b & gpt-oss-20b model card," arXiv preprint arXiv:2508.10925, 2025.
[55] Y. Zhang et al., "Qwen3 Embedding: Advancing Text Embedding and Reranking Through Foundation Models," 2025, technical report arXiv:2506.05176.
[56] W. Wang et al., "MiniLM: Deep Self-Attention Distillation for Task-Agnostic Compression of Pre-Trained Transformers," in Advances in Neural Information Processing Systems, 2020.
[57] Z. Nussbaum et al., "Nomic Embed: Training a Reproducible Long Context Text Embedder," Transactions on Machine Learning Research, 2025.
[58] A. Razavi et al., "Benchmarking Prompt Sensitivity in Large Language Models," 2025, arXiv:2502.06065.
[59] J. Zhuo et al., "ProSA: Assessing and Understanding the Prompt Sensitivity of LLMs," 2024, arXiv:2410.12405.
[60] P. Rajpurkar et al., "SQuAD: 100,000+ Questions for Machine Comprehension of Text," in Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing, 2016.
[61] J. Wei et al., "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models," in Advances in Neural Information Processing Systems, 2022.
[62] C. D. Manning, P. Raghavan, and H. Sch"utze, Introduction to Information Retrieval. Cambridge University Press, 2009.
[63] J. Thorne et al., "FEVER: a Large-scale Dataset for Fact Extraction and Verification," in Proceedings of NAACL-HLT 2018, 2018.
[64] J. Risch, T. Möller, J. Gutsch, and M. Pietsch, "Semantic answer similarity for evaluating question answering models," in Proceedings of the 2021 Workshop on Machine Reading for Question Answering, 2021, pp. 146–152.
[65] N. Reimers and T. Aarsen, "cross-encoder/stsb-roberta-large," https://huggingface.co/cross-encoder/stsb-roberta-large, 2024, sentence-Transformers CrossEncoder model card; trained on the STS Benchmark dataset.
[66] A. Agrawal et al., "Taming Throughput-Latency Tradeoff in LLM Inference with Sarathi-Serve," in 18th USENIX Symposium on Operating Systems Design and Implementation, 2024.
[67] Y. Zhong et al., "DistServe: Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving," in 18th USENIX Symposium on Operating Systems Design and Implementation, 2024.
[68] J. Carbonell and J. Goldstein, "The Use of MMR, Diversity-Based Reranking for Reordering Documents and Producing Summaries," in Proceedings of the 21st Annual International ACM SIGIR Conference on Research and Development in Information Retrieval, 1998.
[69] Y. Yu, W. Ping, Z. Liu, B. Wang, J. You, C. Zhang, M. Shoeybi, and B. Catanzaro, "RankRAG: Unifying context ranking with retrieval-augmented generation in llms," in Advances in Neural Information Processing Systems, 2024.